Лекция 7: Структурные паттерны GoF. Adapter, Decorator, Facade, Proxy. Расширение без наследования

Курс: Объектно-ориентированное программирование 1 (CS2203)
Университет: КазНУ им. аль-Фараби
Преподаватель: Санжар Аубакиров, PhD
Аудитория: Бакалавриат, 2-й курс, специальность «6B06110 Программная инженерия»
Формат: Лекция, 2 академических часа (90 минут)
Методологический нарратив: Order (Сборка подсистем в устойчивые архитектурные структуры)

Введение и инженерный контекст

На прошлой лекции мы изучили, как порождающие паттерны берут на себя контроль за созданием объектов.
Сегодня мы переходим ко второму семейству каталога GoF — **структурным паттернам (Structural Patterns)**. Структурные паттерны отвечают на фундаментальный инженерный вопрос: **как собирать независимые классы и объекты в крупные, устойчивые подсистемы, сохраняя гибкость и слабую связанность?**
В реальных системах (таких как платформа Avtobys) инженерам постоянно приходится: - Интегрировать внешние несовместимые банковские протоколы (Kaspi, Halyk, Visa, Mastercard); - Динамически добавлять к операциям кэширование, аудит, повторы (retry) и замеры времени отклика без раздувания бизнес-кода; - Упрощать сложные распределенные подсистемы для вызывающего клиентского кода; - Контролировать доступ к тяжелым объектам и сетевым соединениям.
Все эти задачи элегантно решаются четверкой ключевых структурных паттернов: **Adapter**, **Decorator**, **Facade** и **Proxy**.

План лекции (90 минут)

№	Время	Тема	Формат
1	00:00–00:20	Паттерн Адаптер (Adapter): интеграция несовместимых интерфейсов и внешних API	Анализ кейсов
2	00:20–00:45	Паттерн Декоратор (Decorator): динамическое расширение поведения без наследования	Теория + код

№	Время	Тема	Формат
3	00:45–01:05	Паттерн Фасад (Facade): сокрытие внутренней сложности подсистем	Архитектурный разбор
4	01:05–01:25	Паттерн Заместитель (Proxy): ленивая загрузка, контроль доступа и сетевые прокси	Практика
5	01:25–01:30	Сравнительный анализ четверки паттернов («Паттерны-обертки») и выводы	Рефлексия

---

1. Паттерн Адаптер (Adapter / Wrapper)

Адаптер преобразует интерфейс одного класса в интерфейс, ожидаемый клиентом. Адаптер обеспечивает совместную работу классов, которая была бы невозможна из-за несовместимости интерфейсов.

Инженерный пример в Avtobys:

Наш валидатор работает с единым внутренним протоколом:

```
class PaymentGateway(Protocol):
    def charge_account(self, account_id: str, amount: Decimal) -> str: ...
```

Банк-партнер предоставил стороннюю SDK-библиотеку LegacyBankXmlClient, у которой метод называется execute_transfer_v2(src_iban, dst_iban, kzt_cents, currency).
Прямое использование этой библиотеки в нашем коде нарушит независимость доменной логики. Мы создаем **Адаптер**:

```
# adapters/bank_adapter.py
from decimal import Decimal
from typing import Protocol


class PaymentGateway(Protocol):
    def charge_account(self, account_id: str, amount: Decimal) -> str: ...


# Адаптер реализует PaymentGateway и оборачивает внешний SDK
class BankSdkAdapter(PaymentGateway):
    def __init__(self, external_sdk_client: LegacyBankXmlClient, transit_company_iban: str):
        self._sdk = external_sdk_client
        self._dst_iban = transit_company_iban

    def charge_account(self, account_id: str, amount: Decimal) -> str:
        # Преобразование данных и форматов
        cents = int(amount * 100)
```

```
        response_xml = self._sdk.execute_transfer_v2(
            src_iban=account_id,
            dst_iban=self._dst_iban,
            kzt_cents=cents,
            currency="KZT"
        )
        return response_xml["transaction_reference_id"]
```

---

2. Паттерн Декоратор (Decorator)

Декоратор динамически подключает к объекту дополнительную функциональность, являясь гибкой альтернативой наследованию.

В чем сила Декоратора?

Декоратор реализует **тот же самый интерфейс**, что и оборачиваемый объект, и хранит ссылку на него. Благодаря этому декораторы можно вкладывать друг в друга как матрешки:

AuditDecorator(CachingDecorator(RateLimitingDecorator(CoreGateway)))

```
# decorators/payment_decorators.py
import time
from decimal import Decimal
from typing import Protocol


class PaymentGateway(Protocol):
    def charge(self, card_id: str, amount: Decimal) -> str: ...


class CorePaymentGateway:
    def charge(self, card_id: str, amount: Decimal) -> str:
        # Реальная тяжелая операция проведения платежа
        return f"TX-{card_id[:4]}-SUCCESS"


# Декоратор аудита и замера времени
class TimingAuditDecorator:
    def __init__(self, wrapped: PaymentGateway):
        self._wrapped = wrapped

    def charge(self, card_id: str, amount: Decimal) -> str:
        start_time = time.perf_counter()
        print(f"[AUDIT START] Списание с {card_id} суммы {amount} KZT")
        try:
            tx_id = self._wrapped.charge(card_id, amount)
            elapsed = (time.perf_counter() - start_time) * 1000
            print(f"[AUDIT SUCCESS] Транзакция {tx_id} за {elapsed:.2f} мс")
            return tx_id
        except Exception as e:
            print(f"[AUDIT ERROR] Сбой списания: {e}")
            raise
```

---

3. Паттерн Фасад (Facade)

Фасад предоставляет простой унифицированный интерфейс к сложной подсистеме из множества классов, скрывая её внутреннее устройство от клиента.
В бортовом терминале валидации прикладывание карты запускает целый комплекс действий: 1. Проверка блокировки карты в локальной базе данных (BlacklistRepository); 2. Расчет персонального тарифа (TariffEngine); 3. Проведение транзакции списания (PaymentGateway); 4. Запись в фискальный журнал (LedgerService); 5. Включение светодиода и подача звукового сигнала (HardwareFeedback).
Без Фасада вызывающий код терминала был бы загроможден вызовами всех 5 подсистем.

```python
# facade/validation_facade.py
class TransitValidationFacade:
    """Единая точка входа для обслуживания пассажира."""
    def __init__(
        self,
        blacklist: BlacklistRepository,
        tariff_engine: TariffEngine,
        gateway: PaymentGateway,
        ledger: LedgerService,
        hardware: HardwareFeedback
    ):
        self._blacklist = blacklist
        self._tariff = tariff_engine
        self._gateway = gateway
        self._ledger = ledger
        self._hw = hardware

    def tap_card(self, card_id: str, bus_id: str) -> bool:
        if self._blacklist.is_blocked(card_id):
            self._hw.signal_error("Карта в стоп-листе")
            return False

        fare = self._tariff.calculate(card_id)
        tx_id = self._gateway.charge(card_id, fare)
        self._ledger.record(tx_id, card_id, bus_id, fare)
        self._hw.signal_success(fare)
        return True
```

Клиентский код теперь состоит всего из одной строки: facade.tap_card(card_id, bus_id)!

---

4. Паттерн Заместитель (Proxy)

Заместитель предоставляет суррогат или объект-заглушку для другого объекта, контролируя доступ к нему.

Виды Proxy:

1. **Virtual Proxy (Ленивая инициализация)**: откладывает создание тяжелого объекта (например, графа остановок всего города на 10 000 точек) до момента первого реального обращения.
2. **Protection Proxy (Защитный)**: проверяет права доступа (роли администратора/инспектора) перед выполнением критических операций.
3. **Remote Proxy (Удаленный)**: инкапсулирует сетевой вызов (gRPC / HTTP REST), делая его для локального кода неотличимым от обычного вызова метода.

Сравнительная таблица паттернов-оберток

Паттерн	Меняет интерфейс?	Назначение
Adapter	Да	Приводит чужой интерфейс к нашему корпоративному стандарту
Decorator	❌ Нет	Добавляет обязанности и поведение без изменения класса
Facade	Да (упрощает)	Объединяет и упрощает работу с группой сложных подсистем
Proxy	❌ Нет	Контролирует доступ, лениво загружает или логирует вызовы

Контрольные вопросы и выводы

1. В чем главное отличие между паттернами Decorator и Adapter?
2. Почему цепочка декораторов соблюдает принцип Single Responsibility (SRP)?
3. Какую проблему решает паттерн Facade в многослойных архитектурах?
4. Чем классический объектно-ориентированный Decorator (GoF) отличается от синтаксического сахара @decorator в Python?

Литература к лекции

1. **Гамма Э. и др.** Паттерны проектирования. – СПб.: Питер, 2020. (Глава 4: Структурные паттерны).
2. **Мартин Р.** Чистая архитектура. – СПб.: Питер, 2021.
3. **Refactoring.Guru**: Структурные паттерны (https://refactoring.guru/ru/design-patterns/structural-patterns).